REPORT

Reviewer report: pyinvoke/invoke#1097: Use

command-line names in missing-argument errors

Run invoke-1097, tested 2026-11-06 11:19 IST by ops/daily_run.py (route source-pair). Proof-of-Change commit

4050401d30fe09fa6a04495c8f41d7203ddf9de8. Report generated 2026-11-06 11:23 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

COULD NOT VERIFY

Recommended action: Re-run Confidence: Low

The run did not produce a usable comparison, so there is no conclusion about the PR. The run did not finish

cleanly, so it says nothing about the PR.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Use command-line names in missing-argument errors (pyinvoke/invoke#1097).

• What we observed: could not verify: claim(s) C1, C2, C4 expect a difference but no observation matches (134

test(s) ran on main, 137 on the PR; 3 new test(s) pass, 135 unchanged (1 of them skipped on both sides); probe

model-p1 is not_adequate (the probe crashed or printed no parsed output on every main-side run; not counted

as ran or adequate); its matching results do not count).

• What you should do: Do not decide yet. Fix the problem under 'How to fix' and run the check again.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 7.5 GB (of 15.6 GB).

• Setup check: free disk 94.7 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: invoke python setup install instructions https://github.com/pyinvoke/invoke

site:github.com/pyinvoke/invoke

• Query: invoke python documentation install https://github.com/pyinvoke/invoke site:github.com/pyinvoke/invoke

• Query: invoke python install requirements https://pypi.org/project/invoke/ https://github.com/pyinvoke/invoke

site:pypi.org

• Source: invoke-common-tasks · PyPI — https://pypi.org/project/invoke-common-tasks/

• Source: invoke · PyPI — https://pypi.org/project/invoke/

• Source: invoke/invoke/runners.py at main · pyinvoke/invoke · GitHub —

https://github.com/pyinvoke/invoke/blob/master/invoke/runners.py

• Source: ImportError: cannot import name yaml2 · Issue #319 · pyinvoke/invoke · GitHub —

https://github.com/pyinvoke/invoke/issues/319

• Source: Dealing with virtual environments · Issue #838 · pyinvoke/invoke · GitHub —

https://github.com/pyinvoke/invoke/issues/838

• Source: Invoke 0.13 breaks on Windows (unavailable default shell) · Issue #371 · pyinvoke/invoke · GitHub —

https://github.com/pyinvoke/invoke/issues/371

• Setup plan: the same packages on main and pr.

• Setup plan: install invocations>=3.3 (listed in dev-requirements.txt) on main and pr.

• Setup plan: install coverage>=6.2,<7 (listed in dev-requirements.txt) on main and pr.

• Setup plan: install releases>=2 (listed in dev-requirements.txt) on main and pr.

• Setup plan: install alabaster==0.7.12 (listed in dev-requirements.txt) on main and pr.

• Setup plan: install pytest-relaxed>=2 (listed in dev-requirements.txt) on main and pr.

• Setup plan: install pytest-cov>=4 (listed in dev-requirements.txt) on main and pr.

• Setup plan: install flake8>=4,<5 (listed in dev-requirements.txt) on main and pr.

• Setup plan: install black>=22.8,<22.9 (listed in dev-requirements.txt) on main and pr.

• Setup plan: install setuptools>56 (listed in dev-requirements.txt) on main and pr.

• Setup plan: install icecream>=2.1 (listed in dev-requirements.txt) on main and pr.

• Setup plan: install mypy==0.971 (listed in dev-requirements.txt) on main and pr.

• Setup plan: install types-PyYAML==6.0.12.4 (listed in dev-requirements.txt) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.6, >=3.9

• Installed invocations 4.0.2 from dev-requirements.txt (listed in dev-requirements.txt) on main and pr.

• Installed coverage 6.5.0 from dev-requirements.txt (listed in dev-requirements.txt) on main and pr.

• Installed releases 2.1.1 from dev-requirements.txt (listed in dev-requirements.txt) on main and pr.

• Installed alabaster 0.7.12 from dev-requirements.txt (listed in dev-requirements.txt) on main and pr.

• Installed pytest-relaxed 2.0.2 from dev-requirements.txt (listed in dev-requirements.txt) on main and pr.

• Installed pytest-cov 5.0.0 from dev-requirements.txt (listed in dev-requirements.txt) on main and pr.

• Installed flake8 4.0.1 from dev-requirements.txt (listed in dev-requirements.txt) on main and pr.

• Installed black 22.8.0 from dev-requirements.txt (listed in dev-requirements.txt) on main and pr.

• Installed setuptools 84.2.1 from dev-requirements.txt (listed in dev-requirements.txt) on main and pr.

• Installed icecream 2.2.0 from dev-requirements.txt (listed in dev-requirements.txt) on main and pr.

• Installed mypy 0.971 from dev-requirements.txt (listed in dev-requirements.txt) on main and pr.

• Installed types-PyYAML 6.0.12.4 from dev-requirements.txt (listed in dev-requirements.txt) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (135 unchanged, 3 runs per side) | pytest-pr-1.log:273 «137 passed, 1 skipped, 5

warnings in 1.29s»

| P-C1 | When a required argument is omitted, the error message shows the Python attribute name instead of the

CLI name. (PR description) | not testable | probe model-p1 has no observation for this claim | no matching log

line

| P-C2 | Two new test cases added for regression coverage fail before the fix. (PR description) | not testable |

probe model-p1 has no observation for this claim | no matching log line

| P-C3 | The control test case passes both before and after the fix. (PR description) | not testable | probe

model-p1 has no observation for this claim | no matching log line

| P-C4 | Real CLI checks confirm that after the fix the missing-argument error uses the CLI name and successful

--param-1/--param-2 calls work. (PR description) | not testable | probe model-p1 has no observation for this claim

| no matching log line

Citation check (WARN): 1 citation(s) verified; 4 statement(s) have no matching log line.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 134 / 137 | | Tests unchanged | 135 | | New tests that pass | 3 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 0 | | Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-p1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p1.py, frozen cc77e0ce80cd

at 2026-11-06T05:53:26.327669+00:00): Test missing-argument error and success for dashed name, unrelated

internal attribute name, and control scenarios.

• P-C1 (PR description): When a required argument is omitted, the error message shows the Python attribute

name instead of the CLI name. Expects the covered observations to differ.

• P-C2 (PR description): Two new test cases added for regression coverage fail before the fix. Expects the

covered observations to differ.

• P-C3 (PR description): The control test case passes both before and after the fix. Expects the covered

observations to stay the same.

• P-C4 (PR description): Real CLI checks confirm that after the fix the missing-argument error uses the CLI name

and successful --param-1/--param-2 calls work. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| probe.error | - | probe could not run | same on both sides: {"c1.a_err": "ModuleNotFoundError", "c2.a_err":

"ModuleNotFoundError", "c2.b_err": "Mod... | probe-model-p1-main-1.jsonl:1 «"observation":

"probe.error"»
probe-model-p1-pr-1.jsonl:1 «"observation": "probe.error"»

Citation check (PASS): 2 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (3.1 s run).

Coverage gate (python, confidence normal): 1 of 1 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| invoke/parser/parser.py | 1 | 1

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested f887e02c1959 of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 1 of 1 changed executable line(s) ran (100%).

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every probe observation cites its output line: 2 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Run did not finish | COULD NOT VERIFY | could not verify: claim(s) C1, C2, C4 expect a difference but no

observation matches (134 test(s) ran on main, 137 on the PR; 3 new test(s) pass, 135 unchanged (1 of them

skipped on both sides); probe model-p1 is not_adequate (the probe crashed or printed no parsed output on every

main-side run; not counted as ran or adequate); its matching results do not count) | Read the setup/build log

under Evidence links; build failures usually need a compiler or a newer nightly.

| Probes ran on both sides | WARN | model-p1 main#1; model-p1 pr#1; model-p1 main#2 | Read the probe-*.log

files; a probe that cannot run is not evidence either way.

| Every claim cites a real log line | WARN | 1 citation(s) verified; 4 statement(s) have no matching log line |

Regenerate the report; if it persists, the logs were edited or truncated.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 1 of 1 changed executable line(s) ran (100%)

| Probes ran on both sides | WARN | model-p1 main#1; model-p1 pr#1; model-p1 main#2

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | WARN | 1 citation(s) verified; 4 statement(s) have no matching log line

| Every probe observation cites its output line | PASS | 2 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-06-check-1118/invoke-1097 DECISION=approve (or reject, approve_with_notes, re_run).

Rerun command
make check PR=https://github.com/pyinvoke/invoke/pull/1097

Evidence links
• Pull request pyinvoke/invoke#1097
• record.json
• pr.diff

• setup-main.log
• setup-pr.log
• git.log
• pytest-main-1.log
• pytest-pr-1.log
• pytest-main-2.log
• pytest-pr-2.log
• pytest-main-3.log
• pytest-pr-3.log
• junit-main-1.xml
• junit-pr-1.xml
• coverage-pytest.log
• coverage-pr.json
• probe-model-p1-main-1.jsonl
• probe-model-p1-pr-1.jsonl
• main run 1 result: pytest-main-1.log:270 «134 passed, 1 skipped, 5 warnings in 1.36s»
• pr run 1 result: pytest-pr-1.log:273 «137 passed, 1 skipped, 5 warnings in 1.29s»

Glossary
| Term | Meaning
| main | The library as it is today, without the PR.
| PR | Pull request: a proposed change to the library.
| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.
| flaky | A test that gives different results when nothing changed; it is ignored as evidence.
| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-06 11:24 IST